SCAFFOLD

Knowledge-State-Driven Adaptive Scaffolding with Proactive Struggle Detection for Programming Labs

FINAL YEAR PROJECT — PROPOSAL DEFENSE DOCUMENT

Team members	Member A — [Full name] (ML Engineer) Member B — [Full name] (Systems Engineer) Member C — [Full name] (Full-stack + Research)
Supervisor	[Supervisor full name], Department of Computer Science
Department	Computer Science, FAST National University of Computer and Emerging Sciences (NUCES)
Document version	v1.0 — Proposal Defense Submission

Table of Contents

Table of Contents 2

Executive Summary 4

1. Complex Computing Challenge & R&D Basis 5

1.1 Problem Statement . . . 5

1.2 Formal Research Question . . . 5

1.3 Literature and Existing-Approaches Review . . . 5

1.4 The Research Gap . . . 6

1.5 Why This Exceeds Course-Project Complexity . . . 6

2. Product / Prior-FYP Comparison & Contribution Clarity 7

2.1 Comparison Against Closest Existing Systems . . . 7

2.2 Prior FAST NUCES FYP Search . . . 7

2.3 Explicit Contribution Statement . . . 8

3. Scope, Requirements & Success Criteria 9

3.1 System Modules . . . 9

3.2 Measurable Acceptance Criteria . . . 9

3.3 Two-Semester Deliverables Timeline . . . 10

3.4 Explicit Non-Goals (Out of Scope) . . . 10

4. Proposed Solution & Technical Approach 11

4.1 Architecture Overview . . . 11

4.2 Algorithms and Models . . . 11

4.3 Infrastructure and APIs . . . 11

4.4 Data Pipeline . . . 12

4.5 Alternatives Considered and Rejected . . . 12

5. POC-Lite / Technical Spike / Feasibility Evidence 13

5.1 POC-Lite Scope (High-Fidelity Interface Prototype) . . . 13

5.2 Required Technical Spikes (Feasibility Evidence Beyond the UI) . . . 13

5.3 Feasibility Evidence Checklist (pre-defense) . . . 13

6. Responsible Tool / API / GenAI Plan 14

6.1 Disclosure Table . . . 14

6.2 GenAI Use Disclosure 14
6.3 Data Licensing 14
6.4 API Cost Budget 15
7. Work Division & Iteration Plan **16**
7.1 Member-Wise Responsibilities 16
7.2 Iteration Cadence 16
7.3 Risk Register 17
8. DEI, Ethical & Legal Issues **18**
8.1 Diversity, Equity & Inclusion Statement 18
8.2 Ethical and Legal Compliance Statement 18
Appendix A — Full Reference List **19**
Appendix B — Draft Informed Consent Form **20**
Appendix C — Classroom Study Design **21**
Appendix D — Glossary **22**

Executive Summary

Scaffold is a closed-loop adaptive tutoring system for undergraduate programming labs that connects three research areas which have, until now, only been studied independently: real-time IDE telemetry, knowledge tracing for programming, and LLM-based hint generation. No existing published system — including the closest comparable work, ClassAid (CHI 2026) — automatically calibrates hint specificity to a live, per-student knowledge model; existing systems either give identical hints to every student or require an instructor to manually switch assistance modes. Scaffold closes this loop: a real-time telemetry pipeline detects struggle, a knowledge tracing engine (Bayesian Knowledge Tracing escalating to a telemetry-enriched Deep Knowledge Tracing model) maintains a live per-concept mastery vector for each student, and a structured prompting pipeline injects that mastery state into an LLM to generate hints at a calibrated disclosure level, filtered through a graduated code-leakage guardrail.

As a secondary, higher-risk research contribution, the team will additionally investigate whether Direct Preference Optimization (DPO) — a lightweight, single-GPU-feasible pedagogical alignment technique that has so far only been applied to mathematics tutoring in the literature — can produce a code-tutoring model whose refusal to leak solutions is baked into model weights rather than enforced only through prompt instructions. This is, to the team's knowledge, the first application of preference-based pedagogical alignment to the code domain, where leakage is a graduated spectrum (signature, pseudocode, partial solution, full solution) rather than the binary leakage of mathematics.

The primary system (Track A) is scoped so that every module — telemetry pipeline, struggle detector, knowledge tracing engine, hint generation service, guardrail layer, and instructor dashboard — can be delivered, integrated, and evaluated within a two-semester timeline by a three-member team, independent of whether the DPO experiment succeeds. The project will be evaluated through a three-condition, within-subjects classroom study measuring transfer-task performance, time to solution, hint dependency, and adversarial leakage resistance.

1. Complex Computing Challenge & R&D Basis

1.1 Problem Statement

Undergraduate programming labs increasingly show a pattern the team refers to informally as "vibe coding": students paste lab problem statements directly into general-purpose AI assistants (ChatGPT, Claude, GitHub Copilot Chat) and submit the returned code with little independent problem-solving effort. This displaces the productive struggle that is known in the learning sciences literature to be necessary for durable skill acquisition, and it is invisible to instructors, who see only a submitted solution and cannot distinguish a student who reasoned through the problem from one who did not.

The population affected is undergraduate students in introductory-to-intermediate programming courses (the team's target deployment is a Data Structures course taught in C++), and secondarily the course instructors and teaching assistants who are responsible for assessing genuine skill development but currently have no tooling to observe it.

1.2 Formal Research Question

> *Does knowledge-state-conditioned, proactively triggered LLM scaffolding — where hint specificity is calibrated using a real-time knowledge tracing model updated from IDE telemetry — produce measurably better learning outcomes (transfer task performance, time to correct solution, hint dependency) compared to reactive, student-model-blind LLM assistance in undergraduate programming labs? As a secondary question: does replacing a prompt-based leakage guardrail with a DPO-aligned model improve pedagogical robustness and resistance to adversarial solution-seeking?*

1.3 Literature and Existing-Approaches Review

The team conducted a structured review of approximately 70 papers (2015–2026) across CHI, SIGCSE, EDM, AIED, ACL, EACL, AAAI, and arXiv, spanning five sub-fields relevant to this project. The table below summarizes the closest and most load-bearing works; the full reference list with links is provided in Appendix A.

Sub-field	Representative works	What they establish	What they do not do
LLM tutoring for programming	CodeHelp (2023), CodeAid (CHI'24), STAP (2025), ClassAid (CHI'26), Closing the Loop (SIGCSE'26)	Prompt-based guardrails can reduce solution leakage; multi-mode hint delivery is viable at classroom scale (54–700+ students).	None maintain a per-student knowledge model; ClassAid requires manual instructor mode-switching, not automated calibration.
Guardrails / leakage prevention	CodeGuard (EACL'26), Answer-Leakage Robustness (ACL'26), Scaffolding Collapse (2026), EduGuardBench (AAAI'26)	Prompt classifiers reduce policy-violating completions by 30–65%; Socratic tutors are vulnerable to adversarial collapse under sustained pressure.	All operate at the prompt/classifier layer; none study the graduated (non-binary) leakage spectrum specific to code.
Knowledge tracing for programming	Code-DKT (EDM'22), srcML-DKT (EDM'25), DPKT (Nature Sci. Rep. '25), Open-Ended KT (EMNLP'22), Struggling-Programmer Detection (2025)	Code-aware neural sequence models predict next-submission correctness from AST/code2vec features with reasonable AUC.	All are offline, batch-mode predictors; none feed their output into a live intervention system.

Sub-field	Representative works	What they establish	What they do not do
IDE telemetry / monitoring	CodeWatcher (ICSME'25), Spark (2026), CodeDive (2025)	Fine-grained keystroke, paste, and compile-event capture is feasible and has been deployed for research analytics.	All are observation-only, feeding instructor dashboards for post-hoc review, not automated real-time pipelines.
Pedagogical alignment (RL/DPO)	TutorRL / PedagogicalRL (ETH, 2025), PEARL (2025), SocraticLM (2024), LearnLM (Google, 2024), Sonkar et al. (EMNLP'24)	GRPO/DPO can train a 7B open-weight model to withhold answers while preserving reasoning ability, on par with proprietary tutors, without human-labeled data.	Every one of these works targets mathematics tutoring exclusively; none address code, where leakage is graduated rather than binary.

1.4 The Research Gap

Two distinct, confirmed gaps motivate this project, one primary and one secondary:

- **Primary gap — the closed loop does not exist.** Knowledge tracing models predict mastery offline; hint generators are student-model-blind; telemetry systems feed dashboards, not intervention pipelines. ClassAid, the closest system, still relies on an instructor manually watching a dashboard and switching hint modes for the whole class rather than an automated per-student calibration. The explicit future-work sections of STAP, Closing the Loop, and ClassAid all call for exactly this integration.
- **Secondary gap — pedagogical alignment has never been applied to code.** Every RL/DPO-aligned tutor in the literature (TutorRL, PEARL, SocraticLM, LearnLM, Sonkar et al.) is trained exclusively on mathematics. Their reward functions treat leakage as binary (the numeric answer was or was not revealed). Code leakage is a spectrum — a function signature, a variable naming convention, a pseudocode skeleton, a partial implementation, and a full solution are all different degrees of disclosure with different pedagogical costs. No existing reward function captures this.

1.5 Why This Exceeds Course-Project Complexity

Dimension	Course-project level	Scaffold's actual requirement
Modeling	Call an LLM API with a system prompt	Implement and empirically compare BKT, DKT, and a novel telemetry-enriched DKT variant with cross-validated AUC reporting
Systems	Single-user CRUD web app	Real-time event pipeline handling 40–60 concurrent students' keystroke/compile/paste streams with sub-200ms capture latency
ML/NLP	Prompt engineering only	Structured prompt construction injecting a live knowledge-state vector, plus an optional DPO fine-tuning experiment with LoRA on a 7B model
Evaluation	Manual demo to supervisor	IRB-approved, three-condition, within-subjects controlled classroom study with a pre-registered primary outcome and statistical hypothesis testing
Security	Basic auth, input validation	A graduated AST-similarity leakage detector and an adversarial jailbreak classifier, red-teamed by the team itself

2. Product / Prior-FYP Comparison & Contribution Clarity

2.1 Comparison Against Closest Existing Systems

Dimension	CodeHelp (2023)	CodeAid (CHI'24)	STAP (2025)	ClassAid (CHI'26)	Scaffold (this project)
Per-student knowledge model	No	No	No	No	Yes — BKT/DKT
Intervention trigger	Student asks	Student asks	Student asks	Student asks or instructor switches	Automatic (struggle detection) + student can ask
Hint calibration	One-size-fits-all	One-size-fits-all	Socratic only	Instructor chooses mode	Auto, from KT state + struggle type
Leakage prevention	Prompt guardrail	Pseudocode-only output	Formalized MVH	Mode-based	Graduated disclosure policy + AST filter (+ optional DPO)
Proactive support	No	No	No	Inactivity detection only	Yes — telemetry-driven
Classroom deployment	Yes (52)	Yes (700+)	No	Yes (54)	Planned (target 30–60)
Cross-session profile	No	No	No	No	Yes — KT persists across labs

2.2 Prior FAST NUCES FYP Search

The team is required to formally search the departmental FYP archive before final submission. The search protocol and template below must be completed by the team with the FYP office and the results inserted here prior to defense.

Search term used	Years searched	Result	Differentiation (if any found)
"AI tutor", "programming assistant"	2020–2026	[To be completed by team]	[To be completed]
"knowledge tracing", "adaptive learning"	2020–2026	[To be completed by team]	[To be completed]
"code plagiarism", "academic integrity"	2020–2026	[To be completed by team]	[To be completed]
"educational chatbot", "LLM tutor"	2020–2026	[To be completed by team]	[To be completed]

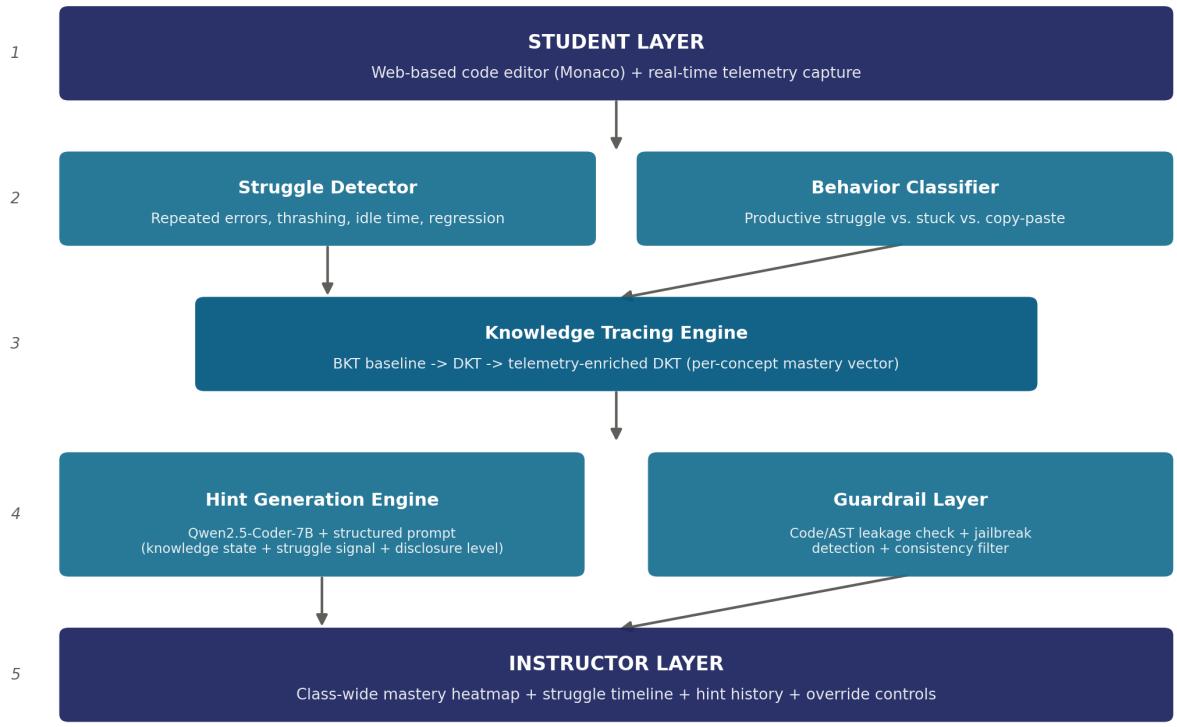
Action item for team: submit this search to the FYP coordination office in Month 1 and populate the table above with actual findings before the proposal is finalized. If no directly overlapping FYP is found, that absence itself should be stated explicitly in the defense.

2.3 Explicit Contribution Statement

We present the first system that closes the loop between real-time IDE telemetry, automated programming knowledge tracing, and knowledge-state-conditioned LLM hint generation, with a graduated code-disclosure policy, evaluated through a controlled classroom study measuring effects on learning outcomes. As a secondary contribution, we present the first application of preference-based pedagogical alignment (DPO) to code tutoring, introducing an AST-similarity-based graduated leakage reward that generalizes prior binary leakage rewards from the mathematics domain to the code domain.

3. Scope, Requirements & Success Criteria

3.1 System Modules

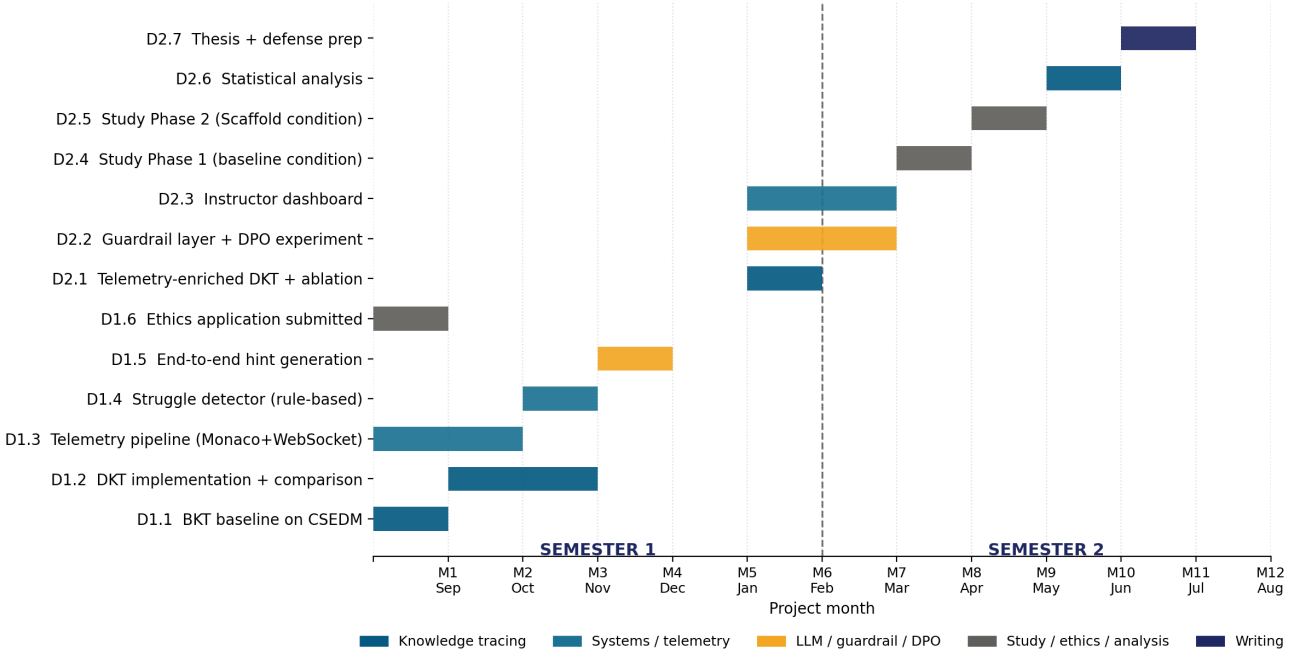


3.2 Measurable Acceptance Criteria

Module	Acceptance criterion
Telemetry pipeline	Captures all event types (keystroke, compile, paste, idle) at <200ms capture latency for 60 concurrent users; zero event loss over a 1-hour sustained-load replay test.
Struggle detector	≥0.75 precision and ≥0.70 recall on the "unproductive struggle" class, validated against hand-labeled pilot segments; classification produced within 30 seconds of a triggering event.
Knowledge tracing (BKT/DKT)	DKT achieves ≥0.70 AUC on held-out students (leave-students-out CV). Telemetry-enriched DKT shows a statistically significant improvement over standard DKT (paired t-test, $p<0.05$).
Hint generation engine	Hint latency <8s for 10 concurrent requests on the deployment GPU. ≥70% of a blind sample of 100 generated hints rated "pedagogically appropriate" by the course instructor.
Guardrail layer	Leakage rate (hints containing executable code with ≥60% AST similarity to the reference solution) below 5% of all generated hints. Jailbreak detection precision ≥0.90 on the CodeGuard evaluation set.

Module	Acceptance criterion
DPO experiment (secondary)	DPO-aligned model shows lower leakage rate than the prompt-only baseline on a held-out adversarial test set, without a statistically significant drop in hint helpfulness ratings.
Instructor dashboard	Class-wide mastery heatmap refreshes within 5 seconds of a new student event; supports drill-down to any individual student's session timeline.
Classroom study	Minimum 30 participants (power analysis for medium effect size, α=0.05, β=0.80, provided in Appendix C); pre-registered primary outcome is transfer-task score.

3.3 Two-Semester Deliverables Timeline



3.4 Explicit Non-Goals (Out of Scope)

- Full GRPO/RL-based pedagogical alignment (Track B) is explicitly out of scope for the core deliverable due to unconfirmed multi-GPU access; it is noted only as a future-work extension.
- Support for multiple programming languages in the first deployment; the system targets a single course (C++ Data Structures) to keep the Knowledge Component taxonomy tractable.
- Automatic grading or academic-integrity enforcement action; Scaffold surfaces struggle and mastery signals to instructors but does not autonomously flag misconduct.

4. Proposed Solution & Technical Approach

4.1 Architecture Overview

The system is organized into five layers (see diagram in Section 3.1): a student-facing web IDE with telemetry capture, a detection layer (struggle detector and behavior classifier), a knowledge modeling layer (the knowledge tracing engine), a generation layer (hint engine and guardrail), and an instructor layer (real-time dashboard). Each layer is a loosely coupled service communicating over a WebSocket event bus and REST APIs, allowing modules to be developed, tested, and evaluated independently before integration.

4.2 Algorithms and Models

Component	Approach	Justification
Knowledge tracing (baseline)	Bayesian Knowledge Tracing (pyBKT)	Interpretable HMM over per-skill mastery; fast to fit; provides an auditable baseline before adding model complexity.
Knowledge tracing (primary)	Deep Knowledge Tracing (LSTM), PyTorch DKT collection	Learns cross-skill dependencies (e.g. loops → recursion) that BKT's per-skill independence assumption cannot capture.
Knowledge tracing (novel)	Telemetry-enriched DKT: concatenates time-on-task, error-type distribution, edit distance, and copy-paste flags to the standard (skill, correct) input	Standard DKT uses only submission correctness; this tests whether behavioral signal improves mastery prediction — the project's primary empirical research question.
Hint generation (Track A)	Structured prompt construction: system rules + problem context + KT mastery vector + struggle signal + disclosure-level instruction, sent to Qwen2.5-Coder-7B via vLLM	Matches the standard approach across the entire literature (CodeHelp, CodeAid, STAP, ClassAid); the novelty is injecting the KT vector, not the prompting mechanism itself.
Guardrail (Track A)	AST-similarity code detector + semantic leakage classifier + adversarial jailbreak classifier (CodeGuard-style)	Output-side filtering is necessary because prompt instructions alone are known to fail under adversarial pressure ("scaffolding collapse").
Alignment experiment (secondary)	Direct Preference Optimization with LoRA on Qwen2.5-Coder-7B, using the ETH PedagogicalRL synthetic-classroom methodology adapted with a graduated AST-based leakage reward	DPO requires no RL training loop or multi-GPU reward-model serving, making it tractable on a single RTX 3090 with the team's current ML experience level.

4.3 Infrastructure and APIs

Layer	Technology	Why
Code editor	Monaco Editor (browser)	Same engine as VS Code; exposes a rich event API for keystroke/paste/cursor capture without a native IDE plugin.
Real-time transport	WebSocket + FastAPI backend	Sub-second event delivery for 40–60 concurrent students; FastAPI's async support handles concurrent connections natively.

Layer	Technology	Why
LLM serving	vLLM with continuous batching, GPTQ INT4 quantization	Benchmarked to serve 50+ concurrent requests at ~11 tok/s each on a single RTX 3090 within a 24GB VRAM budget.
Knowledge tracing service	pyBKT, PyTorch DKT collection, served via FastAPI	Both are open-source, actively maintained, and directly compatible with the ProgSnap2 data format used by the CSEDM benchmark dataset.
Frontend	React / Next.js	Team's existing familiarity; large ecosystem for real-time dashboard components (Recharts) and Monaco integration.

4.4 Data Pipeline

Student keystrokes and compile/paste events stream from Monaco to the backend over WebSocket. Each submission event is scored against test cases and tagged with its Knowledge Component(s) from the course taxonomy. The scored event, together with the accompanying telemetry features, is passed to the knowledge tracing service, which performs a forward pass through the trained model to update the student's mastery vector (a few milliseconds). The struggle detector consumes the raw event stream independently and, on a trigger, requests a hint: the mastery vector, the struggle signal, the student's current code, and the reference solution are assembled into a structured prompt, sent to the LLM, and the raw response passes through the guardrail layer before being displayed. All events are logged to a time-series store for later replay, model retraining, and the classroom study's data analysis.

4.5 Alternatives Considered and Rejected

Alternative	Why rejected
Full GRPO/RL pedagogical alignment (Track B) as the primary deliverable	Requires multi-GPU DeepSpeed training infrastructure not yet confirmed, and RL debugging expertise (reward hacking, entropy collapse) beyond the team's current ML training experience. Retained only as an optional Semester 2 stretch goal if HPC access is confirmed early.
Fine-tuning the hint-generation LLM via full supervised fine-tuning instead of prompting	No existing system in the literature uses full fine-tuning for this task; requires a large labeled hint dataset the team cannot produce at the needed scale within one year.
Using a commercial API (GPT-4/Claude) for production deployment instead of a local open-weight model	Introduces per-request cost at classroom scale, an internet dependency during lab sessions, and data-privacy concerns for student code; used only during development/prototyping.
Training knowledge tracing purely on FAST's own historical grading data	No confirmed historical dataset with the needed granularity exists yet; CSEDM 2021 is used as a pre-training proxy with local fine-tuning/cold-start correction once real data is collected.

5. POC-Lite / Technical Spike / Feasibility Evidence

5.1 POC-Lite Scope (High-Fidelity Interface Prototype)

The POC-lite is a four-screen, high-fidelity, clickable React interface with hardcoded/mock data behind every screen. It demonstrates the full user experience without any of the underlying knowledge tracing, telemetry, or LLM machinery running live. This satisfies the defense requirement for an interface prototype; it is explicitly not claimed as evidence that the underlying algorithms work.

Screen	What it demonstrates	What is mocked
Student IDE view	Monaco editor, problem panel, hint panel with graduated hint levels, live skill-mastery sidebar	Student code, hints, and mastery percentages are all hardcoded
Instructor dashboard	Class-wide concept mastery heatmap, per-student drill-down, hint history timeline	All student and event data loaded from a static JSON file
Student profile / progress report	Per-lab mastery trend, personalized recommendation text, copy-paste incident log	Narrative and chart data are hand-written, not model output
Architecture walkthrough	Animated data-flow diagram naming each real technology at each stage	Pure CSS/SVG animation; no live data flows

5.2 Required Technical Spikes (Feasibility Evidence Beyond the UI)

The committee requires evidence beyond a clickable interface. The team will complete at least one of the three spikes below before the defense, and document the exact output as evidence.

Spike	Task	Evidence to present	Est. effort
A — Knowledge tracing baseline	Load the CSEDM 2021 dataset; fit a BKT model with pyBKT; run model.predict()	Documented AUC score, concept list used, and a screenshot/table of per-student mastery probabilities	4–6 hours
B — Telemetry capture	Embed Monaco Editor; wire keystroke/compile/paste events to a WebSocket backend logging to JSON	Short screen recording showing live event capture while typing/compiling	~2 days
C — End-to-end hint generation	Hardcode a student knowledge-state vector; assemble the structured prompt; call an LLM API; display the returned hint	The exact structured prompt used and the generated hint, for at least 3 different problems	~1 day

Recommendation: complete Spike A, as it is the lowest-effort, highest-signal evidence for the project's core research claim.

5.3 Feasibility Evidence Checklist (pre-defense)

- [ ] POC-lite (4 screens) built and clickable on a laptop/projector.
- [ ] At least one of Spike A / B / C completed and documented with concrete output.
- [ ] RTX 3090 vLLM serving benchmark reproduced locally (or benchmark citations retained as evidence — see Section 4).
- [ ] CSEDM dataset downloaded and its ProgSnap2 schema explored and documented.

6. Responsible Tool / API / GenAI Plan

6.1 Disclosure Table

Tool / Model	Role	Core / Support	License	Fallback if unavailable
Qwen2.5-Coder-7B	Primary inference model for hint generation	Core	Apache 2.0	CodeLlama-7B or Llama-3.1-8B
pyBKT	Bayesian Knowledge Tracing baseline	Core	MIT	Custom HMM implementation
PyTorch DKT collection	Deep Knowledge Tracing model family	Core	MIT	Reimplement DKT from the original paper
vLLM	Concurrent LLM serving with continuous batching	Core	Apache 2.0	Ollama or llama.cpp (lower throughput)
Monaco Editor	Browser-based code editor + telemetry capture	Core	MIT	CodeMirror 6
HuggingFace TRL	DPO training implementation for the secondary experiment	Support	Apache 2.0	Skip DPO experiment; report as future work
ETH PedagogicalRL codebase	Reference methodology adapted for the code-domain DPO experiment	Support	Apache 2.0	Design a bespoke synthetic-classroom pipeline from the paper description
CSEDM 2021 dataset	Pre-training / cold-start data for knowledge tracing	Support	Research data use agreement — see 6.3	LLM-based diagnostic quiz cold-start (see Section 4.5)
Claude / GPT-4 API	Development-time prototyping only; not used in the deployed classroom system	Support (dev only)	Commercial API — usage-based billing	n/a — development convenience only
CodeGuard prompt-classifier methodology	Reference design for the jailbreak-detection component of the guardrail layer	Support	Research methodology (paper), reimplemented	Rule-based jailbreak keyword/pattern filter

6.2 GenAI Use Disclosure

GenAI within the deployed system: Scaffold's core product functionality depends on a generative language model (Qwen2.5-Coder-7B) to produce natural-language hints; this is disclosed as a Core dependency in Section 6.1.

GenAI in the team's own working process: The team has used Claude (Anthropic) as a research and drafting assistant during proposal preparation — specifically for literature discovery, technical feasibility estimation (e.g. GPU throughput benchmarking), and structuring this proposal document. All technical claims sourced this way have been cross-checked by the team against the cited primary sources in Appendix A. This disclosure is provided in case FAST NUCES's GenAI-use policy requires it; the team will confirm the applicable policy with the FYP coordination office and complete any required disclosure form.

6.3 Data Licensing

The CSEDM 2021 Data Challenge dataset is used under its published research data use agreement. The team confirms: (a) the dataset will be used only for model training and evaluation within this project, (b) the raw dataset will not be redistributed, and (c) any published results derived from it will acknowledge the dataset per its citation requirements.

6.4 API Cost Budget

Estimated development-phase cost for Claude/GPT-4 API usage (prototyping, hint-quality comparison baseline, synthetic DPO preference-pair generation): approximately $80–150 total across the two-semester project, based on expected call volume. No commercial API calls are planned during the live classroom study itself; the deployed system runs entirely on the local RTX 3090.

7. Work Division & Iteration Plan

7.1 Member-Wise Responsibilities

Member	Semester 1 milestones	Semester 2 milestones	Primary learning curve
Member A — ML Engineer	D1.1 BKT baseline; D1.2 DKT implementation + comparison	D2.1 Telemetry-enriched DKT + ablation; DPO preference-pair synthesis and LoRA fine-tuning	Knowledge tracing theory (HMM, LSTM), LoRA/DPO, educational-data-mining methodology
Member B — Systems Engineer	D1.3 Telemetry pipeline; D1.4 Struggle detector	D2.3 Instructor dashboard; backend integration of KT service and hint engine	WebSocket/event-streaming architecture, concurrent systems design, Redis
Member C — Full-stack + Research	D1.5 End-to-end hint generation; D1.6 Ethics application	D2.2 Guardrail layer; D2.4/D2.5 classroom study execution; D2.6 statistical analysis	Prompt engineering for pedagogy, LLM serving ops, experimental design and statistics

7.2 Iteration Cadence

- Weekly internal team stand-up (15–20 minutes) to track milestone progress against the timeline in Section 3.3.
- Bi-weekly supervisor check-in with a one-page written status update covering completed work, blockers, and the next two weeks' plan.
- Each module owner (Section 7.1) produces a short integration demo at the end of every month, recorded for the team's own reference and for the mid-year review.
- Schedule slack: Months 5–6 (Semester 1 end) and Month 10 (post-study) are deliberately kept lighter to absorb slippage from earlier milestones.

7.3 Risk Register

Risk	Probability	Impact	Contingency
Ethics approval delayed past the planned study window	Medium	High	Submit application in Month 2, not Month 8; fall back to a convenience sample (team's own batch) with the limitation noted in the thesis if approval is delayed.
HPC/multi-GPU access denied or delayed	Medium	Low (Track A does not require it)	Track A and the DPO experiment both run on the single RTX 3090; HPC is only needed for the out-of-scope Track B extension.
CSEDM dataset is a poor proxy for the FAST course (Python vs. C++, different problem set)	Medium	Medium	Use CSEDM only for pre-training priors; initialize per-student cold-start mastery via a short LLM-graded diagnostic quiz at the start of the deployed course.
Qwen2.5-Coder-7B hint quality is too low for classroom deployment	Medium	High	Fall back to a commercial API for the classroom study only, documenting the local-vs-API quality/cost/privacy tradeoff explicitly in the thesis.
DPO experiment fails to converge or does not improve leakage resistance	Medium-High	Low (secondary contribution only)	Report as a negative result with analysis; the primary Track A system and study remain a complete, defensible thesis on their own.
Struggle detector produces too many false positives, annoying students	Medium	Medium	Tune thresholds against the pilot study's hand-labeled data before the full deployment; allow students to dismiss/snooze proactive hints.

8. DEI, Ethical & Legal Issues

8.1 Diversity, Equity & Inclusion Statement

Diversity and representation. Scaffold is designed to serve all students enrolled in the target course, regardless of background. The team acknowledges that knowledge tracing models trained on external datasets (e.g. CSEDM, ASSISTments) may embed biases from the populations they were collected from, potentially disadvantaging students whose coding style, commenting conventions, or problem-solving approach differs from the training distribution. The team will evaluate knowledge tracing performance disaggregated where possible (e.g. by prior programming exposure, self-reported in an optional survey item) and will report any observed performance disparity explicitly in the thesis rather than omitting it.

Accessibility. The web-based code editor will be tested for keyboard navigability and screen-reader compatibility to the extent feasible within project scope. Students who require accommodations (extended time, alternative interaction modes) may opt out of the behavioral telemetry analysis at their request while continuing to use the assistance features, ensuring no student is excluded from support due to a disability-related need.

8.2 Ethical and Legal Compliance Statement

Informed consent. Students will be informed, prior to any data collection, that their coding interactions are logged, what specific data is collected, how long it is retained, that participation is voluntary, and that non-participation carries no academic penalty. A full consent form draft is provided in Appendix B.

Ethics/IRB approval. The team will submit a formal ethics application to the relevant FAST NUCES committee (or, in its absence, obtain written approval from the Dean of the Faculty) no later than Month 2 of Semester 1. No student data collection will begin before written approval is received.

Data minimization and storage. Collected telemetry (including keystroke-level data) will be stored on university-controlled infrastructure only, not a third-party commercial cloud, with access restricted to the research team. A defined retention period will be specified in the ethics application, after which raw identifiable data will be deleted.

Grade independence. A student's use of Scaffold, or their participation in the classroom study, will have no bearing on their course grade. This will be communicated to students in writing, and the team will request written confirmation from the course instructor that this separation is maintained.

Algorithmic fairness. Knowledge-tracing mastery scores will be used only to calibrate hint content within the system and will not be shared with the course instructor in any form that could influence grading. The team will check for demographic parity in KT predictions wherever demographic information is available and voluntarily disclosed.

Legal compliance. The team will confirm with FAST NUCES whether an institutional data protection policy applies to student interaction data collected under this project, and will ensure the data collection and storage plan complies with it.

Appendix A — Full Reference List

Title	Venue / Year	Link
CodeHelp: LLMs with Guardrails for Programming Classes	Koli Calling 2023	arxiv.org/abs/2308.06921
CodeAid: Balancing Student and Educator Needs	CHI 2024	arxiv.org/abs/2401.11314
STAP: Socratic Tutor for Adaptive Programming	ISAIE 2025	dl.acm.org/doi/10.1145/3775073.3775165
ClassAid: Real-time Instructor-AI-Student Orchestration	CHI 2026	dl.acm.org/doi/10.1145/3772318.3790824
Closing the Loop: Instructor-in-the-Loop AI for Programming	SIGCSE TS 2026	arxiv.org/abs/2510.14457
CodeGuard: Improving LLM Guardrails in CS Education	EACL Findings 2026	arxiv.org/abs/2602.02509
Evaluating Answer Leakage Robustness of LLM Tutors	ACL 2026	arxiv.org/abs/2604.18660
Mitigating Scaffolding Collapse in Socratic Tutors	arXiv 2026	arxiv.org/abs/2607.19371
EduGuardBench: Pedagogical Fidelity & Adversarial Safety	AAAI 2026	ojs.aaai.org/index.php/AAAI/article/view/40399
Deep Knowledge Tracing (original DKT)	NeurIPS 2015	arxiv.org/abs/1506.05908
Code-DKT: Code-based Knowledge Tracing	EDM 2022	arxiv.org/abs/2206.03545
srcML-DKT: Enhanced DKT with Code Representations	EDM 2025	educationaldatamining.org/EDM2025
DPKT: Difficulty-aware Programming KT via LLMs	Nature Sci. Reports 2025	nature.com/articles/s41598-025-96540-3
Open-Ended Knowledge Tracing (OKT)	EMNLP 2022	aclanthology.org/2022.emnlp-main.254
pyBKT: Bayesian Knowledge Tracing Library	EDM 2021	arxiv.org/abs/2105.00385
Detecting Struggling Programmers via Proficiency Taxonomies	arXiv 2025	arxiv.org/abs/2508.17353
CodeWatcher: IDE Telemetry for VS Code	ICSME 2025	arxiv.org/abs/2510.11536
Spark: Real-Time Monitoring of Programming Exercises	arXiv 2026	arxiv.org/abs/2601.22256
Navigating the Landscape of Hint Generation Research (survey)	arXiv 2024	arxiv.org/abs/2404.04728
From Problem-Solving to Teaching Problem-Solving (TutorRL/PedagogicalRL)	arXiv 2025	arxiv.org/abs/2505.15607
PEARL: Training Socratic Tutors with Pedagogically Aligned RL	arXiv 2025	arxiv.org/abs/2605.29582
Pedagogical Alignment of Large Language Models (DPO)	EMNLP Findings 2024	arxiv.org/abs/2402.05000
Towards Pedagogically Aligned LLM Tutors for Math Mistake Remediation	BEA 2026	aclanthology.org/2026.bea-1.10
CSEDM 2021 Data Challenge dataset	NCSU / CSEDM	sites.google.com/ncsu.edu/csedm-dc-2021

Appendix B — Draft Informed Consent Form

Study title: Evaluating Knowledge-State-Driven Adaptive Scaffolding in Programming Labs (Scaffold)
Researchers: [Team member names], Department of Computer Science, FAST NUCES
Supervisor: [Supervisor name]

You are being invited to take part in a research study conducted as part of a Final Year Project. Before you decide whether to participate, please read the following information carefully.

What this study involves. During your regular programming lab sessions, you will use a web-based code editor that may log your coding activity (keystrokes, compile attempts, error messages, and any AI-generated hints you receive) for research purposes. In some lab sessions you will receive hints from a standard AI assistant; in others you will receive hints from the Scaffold system, which adapts to what it estimates you already know.

What data is collected. Timestamps of coding actions, compiler output, hint requests and the hints shown to you, and your responses to a short end-of-lab survey. No data beyond what is described here is collected, and your code submissions are graded exactly as they would be without this study.

Voluntary participation. Your participation is entirely voluntary. Choosing not to participate, or withdrawing at any time, will have no effect on your grade in this course. If you prefer not to have your activity logged, you may still use the lab's standard tools without using the Scaffold system.

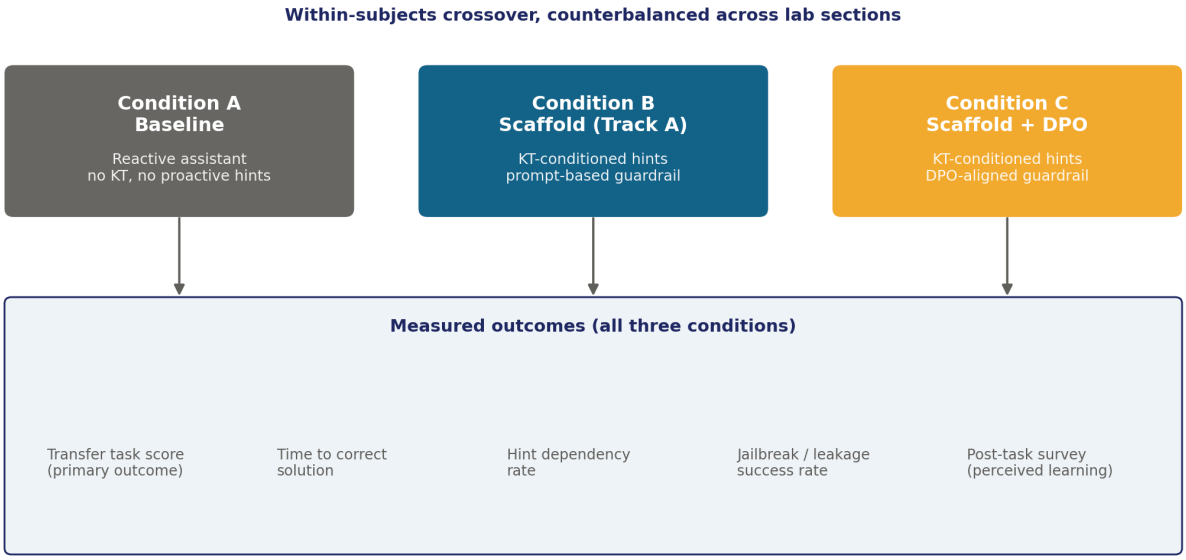
Confidentiality. Your data will be stored on university-controlled systems, accessible only to the research team, and will be anonymized before being used in any thesis, publication, or presentation. Your individual mastery estimates will not be shared with your course instructor in any form that could affect your grade.

Questions or concerns. You may contact [team member email] or [supervisor email] at any time with questions about this study.

Consent. By signing below, you confirm that you have read and understood the above, and voluntarily agree to participate.

Name: _____ Signature: _____ Date: _____

Appendix C — Classroom Study Design



Design: within-subjects crossover across three conditions, counterbalanced across lab sections to control for order effects and instructor-specific variation. **Sample size:** minimum 30 participants, based on a power analysis targeting a medium effect size (Cohen's d = 0.5) at $\alpha = 0.05$ and power $(1-\beta) = 0.80$, which requires approximately n = 27 for a paired-samples comparison; the team targets 30–60 to provide margin for attrition and to support the module-level acceptance criteria in Section 3.2. **Primary outcome:** transfer-task score, i.e. performance on an unseen problem completed immediately after each condition without any AI assistance. **Secondary outcomes:** time to correct solution, hint dependency rate, jailbreak/leakage success rate under adversarial student attempts, and a post-task perceived-learning survey.

Appendix D — Glossary

Term	Definition
BKT	Bayesian Knowledge Tracing — a Hidden Markov Model that estimates the probability a student has mastered a skill from a sequence of correct/incorrect attempts.
DKT	Deep Knowledge Tracing — an LSTM-based neural model that predicts student performance and implicitly learns dependencies across skills.
Knowledge Component (KC)	A discrete, tagged unit of skill or concept (e.g. "recursion", "pointers") that a course's problems are mapped to for knowledge tracing purposes.
AST	Abstract Syntax Tree — a structural representation of code used here to measure similarity between a generated hint's code content and a reference solution.
DPO	Direct Preference Optimization — a supervised alignment technique that trains a model to prefer one response over another from paired examples, without a separate reward model or RL training loop.
GRPO	Group Relative Policy Optimization — a reinforcement learning algorithm used in some prior pedagogical-alignment work; requires multi-GPU on-policy rollout infrastructure not used in this project's core deliverable.
Scaffolding collapse	A failure mode in which a Socratic/hint-based tutor degrades under sustained adversarial pressure and begins revealing direct answers.
Disclosure level	A graduated scale (in this project, Levels 0–4) describing how much of a solution a hint reveals, from a conceptual nudge to a near-complete code skeleton.